

System Design — Day 4

Topic: Async & Consistency — Message Queues, CAP Theorem, Eventual vs Strong Consistency

1. The Problem With Doing Slow Work Inside a Request

Example: a video upload needs to compress, generate thumbnails, scan content, and notify followers — 2 minutes of work. Doing all of this directly inside the request-response cycle causes two problems:

- The user stares at a frozen/loading screen for 2 minutes — a frustrating experience that often leads to them closing the tab or retrying, thinking it failed.
- Requests have timeouts. If the connection is killed mid-process, the work is left silently half-done, with no record that anything is incomplete.

2. The Fix: Message Queues (Async Processing)

Split the work into two parts:

Synchronous (fast, inside the request): save the raw file (e.g. to S3) and immediately respond "Upload received!" — takes 1-2 seconds.

Asynchronous (slow, outside the request): push a message onto a queue ("video 12345 needs processing"). A background worker picks it up and does the heavy work independently of the user's connection.

Result: the user's request-response cycle drops from 2 minutes to 2-3 seconds. The real-world UI pattern this produces is the familiar "Processing..." status seen on YouTube/Instagram right after uploading, before a video is fully live.

3. CAP Theorem

In a distributed system, during a network partition (e.g. the connection between two data centers is cut), you can only guarantee two of these three properties:

Consistency (C): every read gets the most recent write — no stale data.

Availability (A): every request gets some response — never just hangs or refuses.

Partition tolerance (P): the system keeps working even when parts of it can't communicate.

Key insight: network partitions will eventually happen in any real distributed system, so P isn't really optional. The real choice CAP forces is between C and A when a partition occurs.

4. CP vs AP — Choosing the Right Tradeoff

	CP (Consistency + Partition tolerance)	AP (Availability + Partition tolerance)
Behavior during partition	Refuses to answer rather than risk stale/wrong data	Always responds, even if the data might be slightly stale
Example systems	Banking, stock trades, inventory counts	YouTube, Instagram, social feeds

Nightmare scenario if a bank chose AP: a stale balance could let a user withdraw money that's already been spent elsewhere (a "double-spend" problem), because the system serves outdated data instead of refusing to answer.

5. Key Takeaway

Message queues decouple slow background work from user-facing requests, keeping the app responsive. CAP theorem forces every distributed system to consciously choose between strict correctness (CP) and always-on responsiveness (AP) when network partitions happen — the right choice depends entirely on how costly stale data would be for that specific system.